

FLEETGUARD AI

Fleet Assistant Edition — Architecture & Sequence Documentation

Documents what's new in this project: a conversational, read-only Fleet Assistant layered on top of the existing predictive-maintenance pipeline — including a full sequence diagram of a conversation turn.

Covers: fleet_assistant/ (new in this project).

Assumes you've already read: code_documentation_v3.pdf (Python/ML/aerospace fundamentals + the core src_v2/ pipeline) and FleetGuard_AI_Architecture.pdf (train/serve pipeline diagram) — both still apply unchanged. This document covers only what's new.

Contents

1. What's New: the Conversational Fleet Assistant
2. Architecture: a Decoupled Add-On, Not a Rewrite
3. Sequence Diagram: One Full Conversation Turn
4. Component Walkthrough
5. Guardrails & Design Principles
6. Performance Tuning: What Was Slow, and Why
7. How to Run It
8. Known Limitations (Stated Honestly)

1. What's New: the Conversational Fleet Assistant

The core pipeline (src_v2/) already does the hard, safety-relevant work: anomaly detection and RUL prediction, one model pair per component, producing a dashboard and auto-generated work orders. What it doesn't do is let anyone ask it anything — a maintenance planner still has to open an HTML file and read a table.

The Fleet Assistant adds a natural-language layer on top, in a new `fleet_assistant/` folder: a maintenance planner can now ask “*why is AC-014 flagged?*” or “*what needs attention this week?*” in plain English, from a terminal or a simple web chat page, and get an answer grounded in the exact same numbers the dashboard already shows.

Why this matters (not just “because agentic AI is trendy”)

- **Removes the need to learn a UI.** The dashboard's table becomes something you can just ask about instead of having to read and cross-reference by hand.
- **Surfaces context the dashboard doesn't show.** “Has this happened before on other landing_gear?” cross-references the whole fleet automatically.
- **Costs nothing to run.** It works fully offline with zero API cost using deterministic templates, and gets nicer natural-language phrasing for free if a local Ollama model is installed — never a hard requirement.

Design continuity: this follows the same “deterministic core, optional generative layer, graceful fallback” principle established for the rest of this project - see Chapter 5.

2. Architecture: a Decoupled Add-On, Not a Rewrite

The Fleet Assistant does not import any code from `src_v2/`, and `src_v2/` has not been modified at all. It only reads the output files the core pipeline already writes:

<code>outputs_v2/fleet_status_v2.csv</code>	Current health/RUL/anomaly status per unit (from <code>predict_service.py</code>).
---	---

<code>outputs_v2/work_orders_v2.json</code>	Auto-generated work orders (from <code>mro_integration_v2.py</code>).
---	--

<code>outputs_v2/model_metrics_v2.json</code>	Benchmark metrics (from <code>train.py</code>) - used for "how good is the model" questions.
---	---

<code>data_v2/fleet_telemetry_v2.csv</code>	Per-cycle sensor history - used for trend answers ("vibration has been rising").
---	--

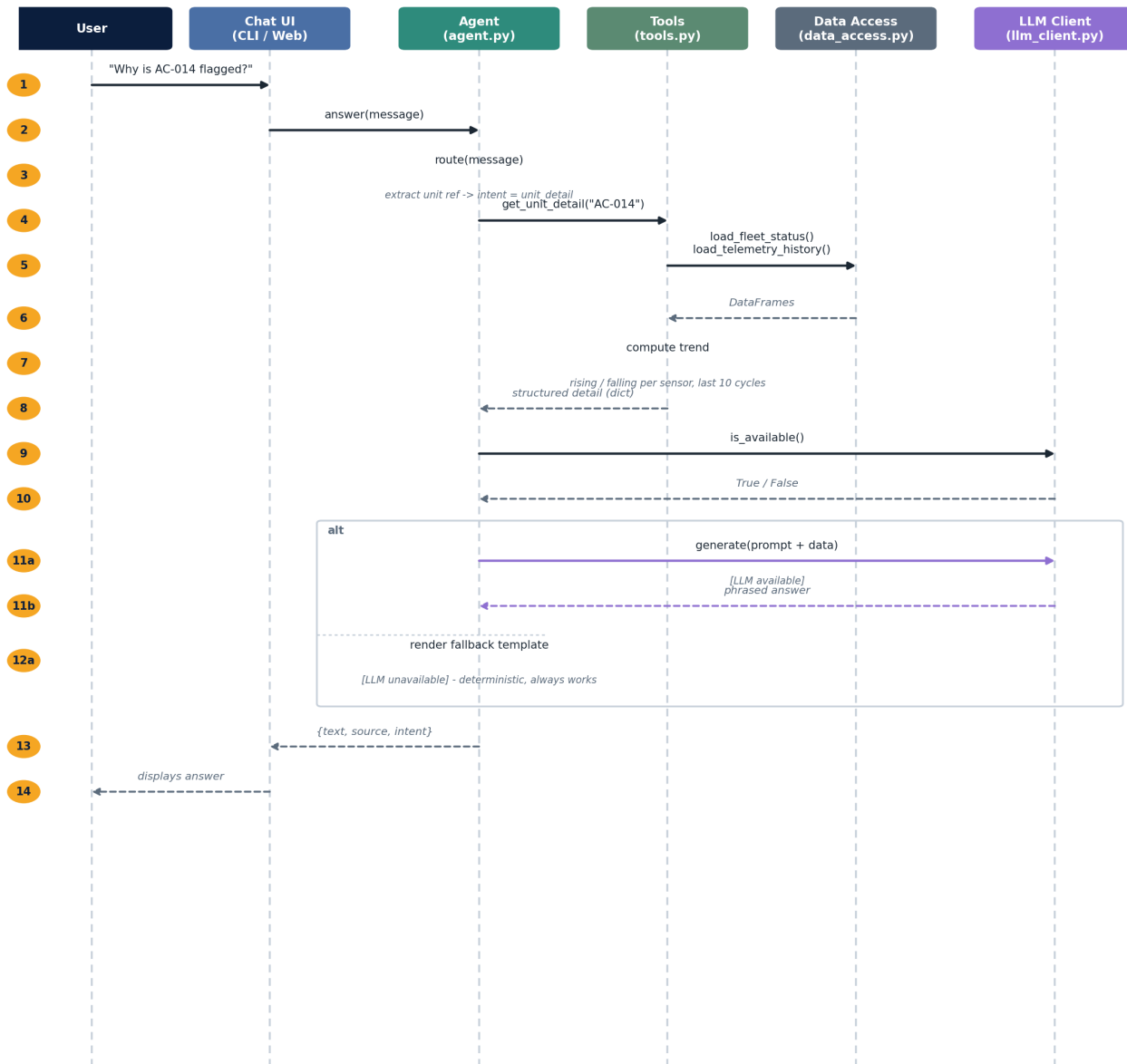
Why decouple rather than integrate directly: this is the same reasoning behind the core pipeline's own train/serve split - two independent pieces that only share data files can be developed, tested, and even deployed separately, without one change risking breaking the other.

The pieces, at a glance

<code>config.py</code>	Paths to the core pipeline's output files; optional LLM settings.
<code>data_access.py</code>	Loads those files into pandas DataFrames / dicts.
<code>tools.py</code>	Read-only functions: fleet summary, top priority, unit detail + trend, similar incidents, work-order lookup.
<code>agent.py</code>	Figures out which tool(s) answer a question, then phrases the answer.
<code>llm_client.py</code>	Optional call to a free, local Ollama model; returns None on any failure.
<code>chat_cli.py</code>	Terminal chat loop.
<code>chat_web.py</code> + <code>static/chat.html</code>	A one-page Flask web chat UI.

3. Sequence Diagram: One Full Conversation Turn

The diagram below traces every step of a real question — “Why is AC-014 flagged?” — exactly as agent.py actually executes it. Dashed arrows are returns; the boxed **alt** section shows the two possible paths depending on whether a local LLM is reachable.



3. Sequence Diagram — Step-by-Step Explanation

- **Steps 1–2:** the user's question reaches `agent.answer()` unchanged - no preprocessing happens in the UI layer.
- **Step 3:** `route()` is pure Python pattern matching (regex + keyword lists) - no LLM call is needed just to figure out what kind of question this is.
- **Steps 4–8:** `tools.get_unit_detail()` does all the real work: load the current status, load historical readings, compute a simple rising/falling trend per sensor.
- **Steps 9–10:** a fast, short-timeout reachability check - if Ollama isn't running, this returns False almost instantly rather than hanging.
- **Steps 11a–11b (LLM available):** the LLM receives the already-computed structured data as its only source of truth and just rephrases it in natural language.
- **Step 12a (LLM unavailable):** a plain Python f-string template renders the exact same data into readable sentences - no external call, cannot fail.
- **Steps 13–14:** the response (with a "source" field telling you which path was taken) flows back to the UI and is displayed.

The one constant across both branches of the alt box: the underlying numbers never change. The LLM only ever rewords data `tools.py` already computed - it cannot introduce a fact that isn't in that data.

4. Component Walkthrough

4.1 data_access.py + tools.py — the read-only data layer

```
def draft_work_order(unit_id_or_aircraft):
    orders = load_work_orders()
    matches = [wo for wo in orders if wo['unit_id'].upper() == key ...]
    if not matches:
        return {"has_draft": False, "query": unit_id_or_aircraft}
    return {"has_draft": True, "work_orders": matches}
```

Every tool function returns plain dicts/lists built from already-existing data - never a number invented or estimated on the spot. `draft_work_order()` is a good example: it doesn't build a new work order, it looks up whether the core pipeline already made one, and returns it clearly labeled as not yet sent.

4.2 agent.py — routing and response generation

```
UNIT_ID_PATTERN = re.compile(r"\bAC-\d+(?:-[A-Za-z_]+\d+)?\b", re.IGNORECASE)

def route(message):
    unit_ref = _extract_unit_ref(message)
    if unit_ref:
        return "unit_detail", tools.get_unit_detail(unit_ref)
    ...
    return "summary", tools.get_fleet_summary() # default
```

Routing is intentionally simple keyword/regex matching, not an LLM call - this keeps it fast, free, and fully testable without any external dependency (see `tests/test_agent.py`). A specific aircraft/unit reference always takes priority over generic keywords, so a question naming a specific part gets an answer about that part rather than a generic fleet summary.

Before building the LLM prompt, `_summarize_for_llm()` trims large results (e.g. several matching units, several work orders) down to the most relevant one or two - the deterministic template fallback still always uses the full, untrimmed result. This exists purely for response speed on a local model; see Chapter 6.

4.3 llm_client.py — optional, never required

```
def generate(prompt, model=OLLAMA_MODEL):
    resp = requests.post(f"{OLLAMA_HOST}/api/generate",
        json={"model": model, "prompt": prompt, "stream": False,
            "options": {"num_predict": LLM_MAX_OUTPUT_TOKENS}},
        timeout=(LLM_CONNECT_TIMEOUT_SECONDS, LLM_TIMEOUT_SECONDS))
    ...
    except requests.exceptions.RequestException:
        return None
```

Every possible failure (Ollama not installed, not running, model not pulled, network hiccup) is caught and turned into a plain `None` - `agent.py` treats `None` exactly like “no LLM available” and falls back to a template. Nothing in this codebase ever crashes because a generative model wasn't reachable. `is_available()` is also cached for `LLM_AVAILABILITY_CACHE_SECONDS`, and `num_predict` bounds reply length - both added after a real performance investigation, covered in full in Chapter 6.

4.4 chat_cli.py and chat_web.py — two interchangeable front ends

Both call the exact same `agent.answer()` function - the CLI and the web UI are just two different ways of displaying the same conversation. Adding a third front end (e.g. a Slack bot) would mean writing a new, thin adapter, not touching `agent.py`, `tools.py`, or `data_access.py` at all.

5. Guardrails & Design Principles

- **Propose, never execute.** No tool sends a work order, changes a schedule, or writes any file. `draft_work_order()` only surfaces what the core pipeline already drafted - a human still has to act on it through the pipeline's normal flow.
- **Numbers come only from tools, never from the LLM.** The LLM (when used) is handed already-computed data and asked only to reword it - it cannot introduce a number, unit, or fact that isn't already in that data.
- **Graceful degradation, always.** Every LLM call path has a deterministic fallback. The assistant is fully functional with zero installed AI model.
- **Decoupled by file, not by import.** The assistant depends only on the core pipeline's output file formats, never on its internal code - the same production pattern as the train/serve split in `src_v2/`.

6. Performance Tuning: What Was Slow, and Why

Early testing surfaced real, measured latency problems in the LLM-phrased path - not "LLMs are just slow" in the abstract, but three specific, fixable causes. This section documents what was found and why the current defaults in `config.py` are set the way they are, so a future change doesn't accidentally reintroduce them.

6.1 Bug: resolving "localhost" cost ~1.5s on every message

On this Windows setup, connecting to `http://localhost:11434` tried IPv6 (`:::1`) first, failed, then fell back to IPv4 - adding a fixed ~1.5 second delay to *every single* reachability check and LLM call. Switching the default host to `127.0.0.1` skips that resolution entirely (measured: 1.51s → 0.01s for the same reachability check).

6.2 Bug: unbounded generation timed out and still fell back to a template

Without a reply-length limit, the model would sometimes generate a long-winded response that took longer than the read timeout - meaning the request failed *after* paying nearly the full wait, then fell back to a template anyway. Setting `options.num_predict` in the Ollama request bounds how many tokens the model may generate, so a reply finishes (or fails) well inside the timeout instead of running past it for nothing.

6.3 Measured: this model generates at roughly 10 tokens/second on CPU

Direct timing against the local Ollama server (no GPU, this test machine) showed consistent generation speed regardless of prompt content: about 7 tokens generated in ~0.65 seconds. At that rate, an unbounded or overly generous `num_predict` can take far longer than expected purely from CPU throughput - `LLM_MAX_OUTPUT_TOKENS = 90` (`config.py`) was chosen to reliably finish (~9s worst case) inside `LLM_TIMEOUT_SECONDS = 15`, with margin.

6.4 Fix: trim large data payloads before building the prompt

Questions that match several units or several work orders (e.g. "why is AC-014 flagged?" matching all 4 of its components) produce a longer input prompt, which itself takes the model measurably longer to process before it can start generating a reply. `agent._summarize_for_llm()` trims these down to the single most urgent match for the LLM prompt only - the instant template fallback still always uses the complete, untrimmed data, so no information is lost when no LLM is available.

6.5 Fix: cache the reachability check

`llm_client.is_available()` now caches its result for `LLM_AVAILABILITY_CACHE_SECONDS` (default 30s) instead of making a fresh network call on every message - Ollama's up/down status doesn't change mid-conversation in practice, so re-checking every time was pure overhead.

Net effect measured on this machine: before these fixes, replies took 8-12+ seconds with frequent silent timeouts that fell back to a template anyway. After, replies consistently take 3-10 seconds and actually succeed with real LLM phrasing.

6.6 If it still feels slow

- Try a smaller model: `ollama pull llama3.2:1b` and set `OLLAMA_MODEL=llama3.2:1b` - meaningfully faster generation at some cost to reply quality.
- Use template mode: skip installing Ollama entirely. Answers are near-instant and use the exact same underlying data, just with plainer, non-LLM wording.
- CPU token-generation speed is the fundamental limit once the fixes above are in place - a GPU or a smaller model are the only ways to meaningfully go faster, not further code changes to this project.

7. How to Run It

7.1 One-time setup

```
cd fleet_assistant
pip install -r requirements_assistant.txt
```

Optional, for LLM-phrased answers instead of templates: install Ollama separately and pull a small model (e.g. `ollama pull llama3.2`). Not required - everything below works without it. See Chapter 6 for what to expect from response times.

7.2 Terminal chat

```
python chat_cli.py
```

7.3 Web chat

```
python chat_web.py
# then open http://localhost:5000
```

7.4 Run the tests

```
python -m unittest tests.test_tools tests.test_agent -v
```

Prerequisite for real (non-fixture) answers: the core pipeline must have already been run at least once (`train.py` then `predict_service.py` from `src_v2/`, per `FleetGuard_AI_Architecture.pdf` page 2) so `outputs_v2/` contains real data.

8. Known Limitations (Stated Honestly)

- **Routing is keyword/regex matching, not true language understanding.** Unusually phrased questions may fall through to the generic fleet-summary default instead of the intent you meant. This is a deliberate simplicity/reliability trade-off, not an oversight - it never fails or hallucinates an intent.
- **“Similar incidents” compares against the current fleet snapshot, not a real incident/root-cause database.** A production deployment would maintain a proper historical incident log.
- **Telemetry trend history uses the training-time snapshot CSV,** not a live telemetry warehouse - fine for this project's batch-oriented design, but a real deployment would query a proper time-series store.
- **The Flask dev server (chat_web.py) is explicitly not production-hardened** (no auth, no HTTPS, single process) - it's built for local demo use, matching the rest of this project's scope.

See [code_documentation_v3.pdf](#) for everything about the core pipeline this assistant sits on top of, and [FleetGuard_AI_Architecture.pdf](#) for the train/serve pipeline diagram.